SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction

This section introduces Z.AI Emergency Portal, a solution designed to address the limitations of current disaster response mechanisms, particularly in flood-prone regions in Malaysia.

Z.AI Emergency Portal focuses on enabling reliable communication, intelligent evacuation and efficient rescue coordination, even under extreme conditions where existing infrastructure such as internet connectivity and power supply may fail.

1.1. Purpose

This System Analysis Document outlines the technical scope, system architecture, and key design decisions behind the development of Z.AI Emergency Portal. It provides a structured understanding of how this system operates across different disaster scenarios, ensuring clarity for development, testing, and future scalability.

So, the key element of system that has been covered are as followed:

1. **Architecture**
   Z.AI Emergency Portal adopts an offline-first, distributed architecture that combines mobile-based application, AI decision engine and decentralized communication via mesh networking (Bluetooth and WiFi). This system is designed to continue its operation even when parts of the infrastructure fail.

2. **Data Flows**
   Z.AI Emergency Portal manages data flow across both online and offline environments. Firstly, user input is collected via mobile devices and is processed by AI. In normal (online) condition, data flows through backend servers and is stored for coordination, while in offline conditions the data is transmitted using store and forward mechanisms across nearby devices. As a final output, structured rescue information is delivered to command centers.

3. **Model Process**
   The system model represents the end-to-end workflow of core functionalities:
   1. Disaster alert is triggered
   2. AI generates evacuation guidance
   3. User interacts via minimal UI for route update or SOS
   4. AI processes and prioritizes emergency cases
   5. Rescue teams receive structured tasks that have been ranked according to priority
   6. System continues tracking /predicting user status

© UMHackathon 2026

4. **Role of Reference**

This document serves as a technical reference for:

a. **Developers**: Understand system structure and implementation logic
b. **Designers**: Align UI/UX with system constraints
c. **Testers**: Validate system behaviour under different scenarios

This ensures all stakeholders share a clear, unified understanding of how Z.AI Emergency Portal operates.

1.2. Background

Flood-prone regions in Malaysia frequently experience severe disruptions during disaster events. Current disaster response processes are commonly manual and reactive, dependent on stable internet & communication infrastructure and are limited by unstructured and delayed information from victims. In many cases, victims are unable to communicate effectively due to panic, network disconnections or device limitations. As a result, rescue teams often operate with incomplete situational awareness, which lead to delays and inefficiencies in saving lives.

1. **Previous Version**

Existing disaster related solutions typically operate as centralized mobile or web platforms which depend on internet connectivity and manual user reporting. Common features include Broadcast alerts, static evacuation guidelines and emergency hotlines.

However, these systems have key limitations:

a. Cannot function when network connectivity is lost
b. Require users to actively input clear information
c. Lack real-time prioritization and intelligent decision support

2. **Changes in Major Architectural Components**

Z.AI Emergency Portal introduces some key architectural shifts compared to traditional systems:

a. Centralized to a distributed system that combines cloud + device level processing + mesh networking.
b. Online to offline-first design, since traditional solutions require the internet to function but Z.AI Emergency Portal continues operating using local processing and Store & Forward communication.
c. Manual input to AI-Assisted Decision System, where AI interprets users' minimal inputs instead of clearly describing their situations.

© UMHackathon 2026

d. Pre-existing data to Real time Adaptive System, instead of fixed evacuation routes, Z.AI Emergency Portal features dynamic updates based on user reports and environmental changes.

3. **New capabilities introduced**
   a. Mesh Network Communication: Data transmission without internet
   b. AI-based SOS structuring & prioritization: Messy inputs become actionable rescue data
   c. The "Dead Phone" Protocol: Maintain operation under low battery/device failure
   d. Last Known Position & Predictive Tracking: Allows rescue action even after disconnection
   e. Offline navigation with Hazard adaptation: Ensures safe evacuation routes when connectivity is unavailable

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Residents living in flood-prone area /Victims	Receive alerts, follow evacuation routes, send SOS signals and update status	Simple and fast interaction, reliable guidance, ability to communicate without internet, assurance of being located and rescued
Rescue Teams / First responders	Receiving SOS requests, prioritize rescue missions and locate victims	Accurate and structured information, clear prioritization (P0–P3), real-time updates, efficient navigation to victims
Government Agencies (e.g. Jabatan Meteorologi Malaysia)	Issue disaster warnings, oversee coordination and manage response strategies	Reliable system integration, real-time situational awareness, scalable and secure platform
Development team	Build the platform and maintain the system	Clear API usage contracts, codebase is scalable and maintainable, well-defined system flows
Command Center Operators	Monitor dashboard and manage rescue dispatch	Ensuring real time, structured information and ability to override AI decisions when needed

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview

Type	Details
System	Mobile App (IOS & Android)
Architecture	Offline-first, distributed client-server architecture with AI service layer and mesh networking support

Z.AI Emergency Portal is structured as a hybrid distributed system containing user mobile application, command center dashboard, backend API layer, LLM Service Layer and device to device network. For the core architecture flow, the mobile app will collect user input (SOS) and sensor data (GPS and battery), and the data will be processed locally during offline mode or sent to backend API (online mode). Then the backend API will interact with LLM (GLM) for structuring and prioritization and the database will store rescue data. Lastly, the output will be delivered to the Command Center Dashboard.

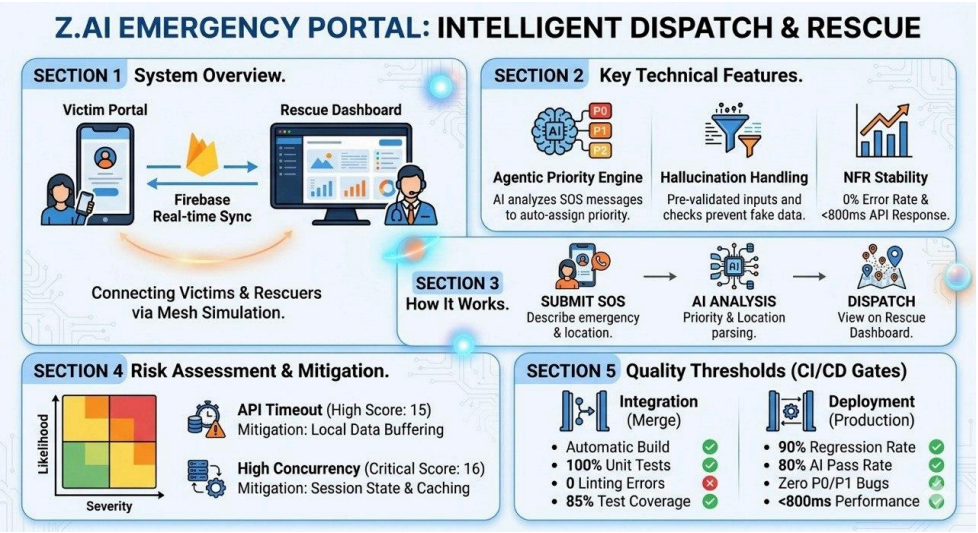
2.1.2 LLM as Service Layer

2.1.3 Dependency Diagram

a. Input sources from the structured button input or unstructured voice/text will be cleaned & normalised, truncated to essential fields and converted into structured prompt format and sensor will be sent to GLM.
b. The prompt includes the user's profile containing basic info, current situation such as location, water level, battery, and context such as disaster state and nearby hazards.
c. The GLM will return a structured SOS report based on priority level (P0-P3) and risk assessment, the system then stores the responses in the database and sends them to the command dashboard.
d. Max input size is enforced before API call, if exceeded there will be truncation and chunking if needed. This ensures cost control and fast response.

© UMHackathon 2026

2.2.2. Sequence Diagram



2.2. Technological Stack

2.2.1. Frontend : Python (Streamlit) – Chosen for rapid MVP prototyping to deliver a responsive, minimalist emergency interface across all devices.

2.2.2. Backend : Python (Streamlit) – Integrated server-side logic and session handling for fast deployment of AI processing tasks.

2.2.3. Database : Firebase Firestore – NoSQL document-based storage selected for its real-time synchronization and native offline persistence capabilities.

2.2.4. Communication: Simulated Mesh Network – Implementation of a local store-and-forward queue to mimic resilient device-to-device SOS relay.

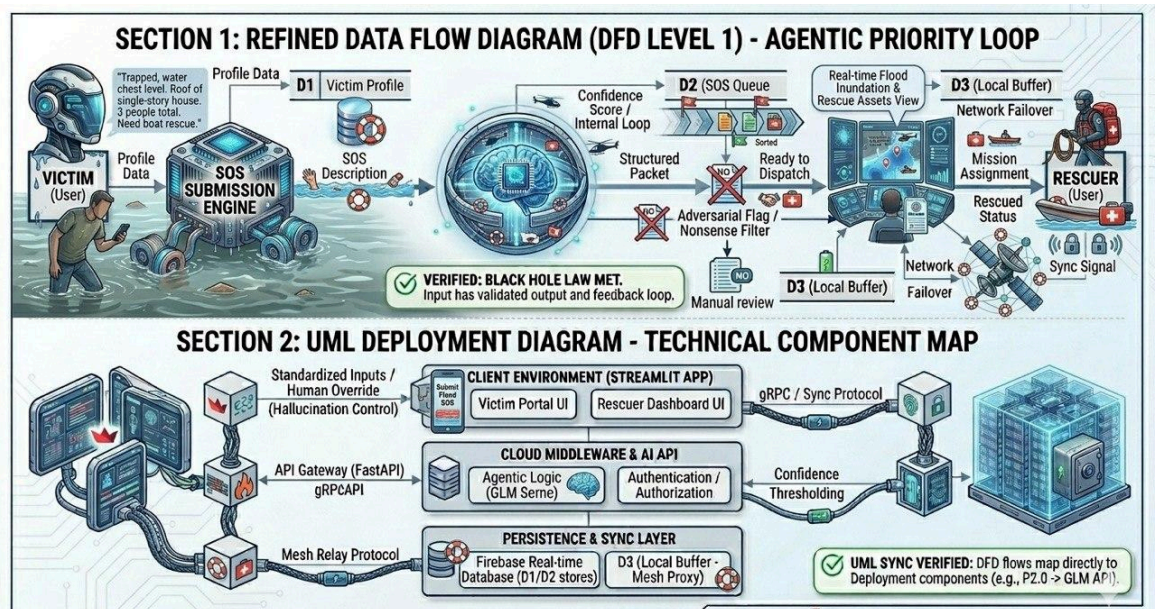
2.2.5. Cloud/Deployment: Streamlit Cloud & Firebase – Leveraged for scalable hosting and secure database management during the hackathon period.

© UMHackathon 2026

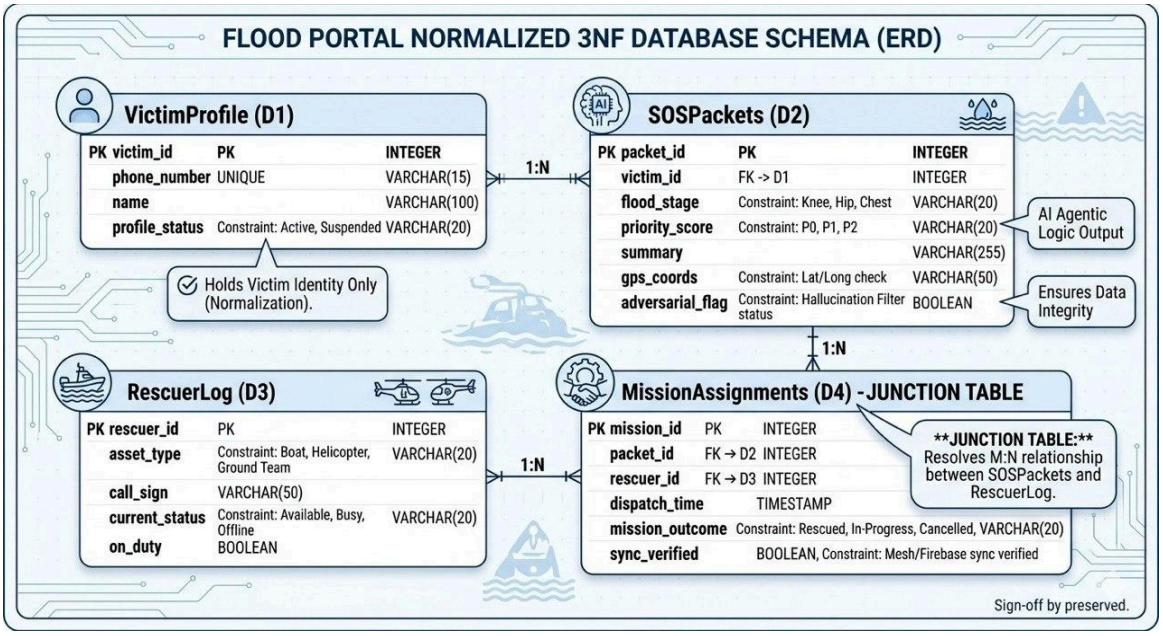
2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema



© UMHackathon 2026

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product

3.1.1. Core features that the team is going to build and do a demo by the end of the UMHackathon 2026.

#	Features	Description
1	One Tap SOS system	The system features a button-based emergency reporting which does not require typing and converts users input (location, water level, and situation) into structured rescue data.
2	Command Center Dashboard	All incoming rescue requests are displayed as structured task cards and automatically sorted by priority, time and status. This enables efficient rescue dispatch and tracking.
3	AI Powered Priority Engine	The system automatically assigns rescue priority level (P0-P3) and processes sensor data and user input, ensuring the most critical cases are rescued first.

© UMHackathon 2026

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Scalability	System must support large numbers of users during disaster peaks in regions	Modular architecture with cloud-based backend services; ability to scale AI processing and data handling independently; lightweight SOS packets reduce system load
Reliability	System must continue functioning even under network failure, low battery, or partial system breakdown	Offline-first design; mesh network (Bluetooth/WiFi); store-and-forward mechanism; fallback to minimal SOS mode
Maintainability	System components must be easy to update and manage without affecting the entire system	Separation of components: UI, AI processing, communication layer; modular design allows independent updates and testing
Latency	Critical operations such as SOS generation and alert display must respond within 3-5 seconds under normal conditions	Event driven AI processing; lightweight structured data instead of heavy inputs; timeout handling with fallback UI states
Cost Efficiency	Token usage per session must remain minimal and predictable to allow scaling	Button-based inputs reduce token size; event-triggered AI calls; caching repeated patterns; truncation of non-essential input data
Energy Efficiency	System must minimize battery usage during disaster event	Low power mode activation; disabling non-essential UI elements; reduced background processing; emergency-only data transmission
Availability	System should maintain core functionality even without internet connectivity	Offline maps; mesh communication; local processing; graceful degradation to basic SOS mode

© UMHackathon 2026

3.3. Out of Scope / Future Enhancements

Several advanced capabilities are identified as part of Z.AI Emergency Portal's long-term vision, but are not included in the current implementation.

a. Satellite Communication Backup to enable direct to satellite SOS transmission
b. Multi-Disaster Support (e.g. Earthquake, wildfires)
c. Community based response network to allow nearby civilians assist in rescue and receive localized alerts

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

4.1.1. Grayscale Rollout & A/B Testing

Z.AI Emergency Portal adopts grayscale (phased) rollout strategy to ensure system reliability before full deployment, especially in high risk environments. New features (such as improved SOS prioritization and routing logic) are initially released to 5-10% of users in selected regions. The system performance is monitored based on SOS success rate. AI accuracy and system latency. If the performance is stable then gradually expand to 100 % of users. Z.AI Emergency Portal also applies A/B testing to optimize critical functionalities. Evaluation Metrics is based on evacuation success rate, time taken to reach safe zones and user interaction efficiency.

4.1.2. Emergency Rollback (ER) & Golden Release

Z.AI Emergency Portal maintains a Golden Release, which is the last stable and verified version of the system. If critical anomalies are detected, the system automatically triggers Emergency Rollback and reverts to Golden release.. Conditions are triggered if there are SOS system failure, AI misclassification spike, Bcck API downtime and Data corruption or inconsistency. As an action to counteract it, the system will disable faulty features, restore stable services and notify system operators. This ensures continuous availability of core life-saving functions.

4.1.3. Priority Matrix

Priority	Trigger Condition	Action Taken
Critical	Users unable to send SOS or system crash	Immediate rollback + switch to basic SOS mode
High	AI prioritization error rate >10%	Disable AI module and fallback to rule based logic
Medium	Mesh communication delays / partial failure	Retry transmission + store and forward continues
Low	UI bugs / minor display issues	Fix in the next update cycle

© UMHackathon 2026

4.2. Monitoring

4.2.1. Z.AI Emergency Portal implements real-time monitoring to ensure system health and rapid response. The key metrics are:

1. API response time
2. SOS delivery success rate
3. AI processing latency
4. Error rates
5. Connectivity Status

Alert will be triggered when:

1. Error rate exceeds threshold
2. SOS signals fail repeatedly
3. Latency spikes beyond acceptable limits

The system will response:

1. Automatic fallback activation
2. Logging and diagnostics
3. Notification to operators

4.3. Assumptions

Our key assumptions are:

a. Users have installed the app prior to disaster and already enable GPS, Bluetooth and necessary permissions

b. External data sources (weather condition from Jabatan Meteorologi Malaysia) are operational

c. There is sufficient device density for mesh networking

d. Users have at least minimal battery levels before disaster occurs

© UMHackathon 2026

4.4. External Dependencies

Tools	Purpose	Risks
Z Ai GLM API	Core AI engine for SOS structuring & prioritization	High risk: API failure Mitigation: Fallback to rule based processing
Offline Map Data	Navigation and evacuation routing	Medium risk :Outdated maps Mitigation: Periodic updates required
Bluetooth and WiFi Direct	Mesh Network Communication	Medium Risk: Limited range Mitigation: Store and Forward Mechanisms
Cloud Backend (API +DB)	Data Storage and Coordination	High risk: Server Downtime Mitigation: Offline first fallback ensures continuity

© UMHackathon 2026

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline

1. **Day 1-2: Planning and Ideation**
   The team identified problems, brainstormed solutions, defined scope and core features and began initial system architecture design.

2. **Day 3: System Design**
   Finalized architecture, defined data flow, user journey and AI workflow.

3. **Day 4-7: UI/UX & Prototyping**
   Designed a minimalist emergency interface, which focuses on button-based interaction and dark mode to save power and successfully build a prototype.

4. **Day 8: Core Development and Finalization**
   SOS system logic and AI processing simulation were implemented, carried out testing and refinement to improve system reliability and fallback mechanisms.

5. **Day 9: Documentations and Pitch Preparation**
   Finishing PRD, SAD and QATD documentations and preparing slides for pitching.

6. **Day 10: Project Submission**
   Filming pitching video and submitting all the documentations required.

© UMHackathon 2026

5.2. **Team Members Role**

Team Member	Role	Responsibilities
Lew Zi Yee	Project Lead / Full-Stack & AI Integration	Led overall project direction and system design; conceptualized Z.AI Emergency Portal workflow and architecture; developed core system logic; implemented backend functionalities; integrated the system with the Z AI model
Shieh Yu Yang	UI/UX Designer & Frontend Support	Designed user interfaces for both end-users and rescue command center; ensured intuitive and emergency-friendly UI/UX; assisted in frontend/backend development; prepared quality assurance testing documentation (QATD) ;created visual materials and illustrations;
Elisha Wong Rong Sheng	Documentation & System Analyst	Prepared System Analysis Document (SAD) and Product Requirement Document (PRD); structured technical and functional specifications; supported overall project communication and presentation

5.3. Recommendations

Introduce redundant communication channels such as satellite backup. For repeated patterns, data caching mechanisms can be implemented and ensure the system still works under a limited circumstance.

© UMHackathon 2026